Resilient Anomaly-Aware Building Access Control

D7065E, Embedded Intelligence at the Edge - Architecture Document

Stefan Höglund (stehog-6)

Contents

1 Summary **2**

2 Use case and building context **2**

3 Requirements **3**

4 Architecture **4**
4.1 Context diagram (C4 Level 1) 4
4.2 Container diagram (C4 Level 2) 5
4.3 Component diagram (C4 Level 3) 7
4.4 Design decisions and alternatives 8

5 Interfaces and communication **9**

6 Simulation **10**

7 Autonomous decision-making and data pipeline **12**
7.1 The autonomous service 12
7.2 The data pipeline 12

8 Behaviour **13**
8.1 Key scenario: successful passage (dynamic diagram) 13
8.2 Door-control lifecycle (state diagram) 14

9 Deployment **15**

10 Test plan and verification **16**

11 Evaluation **17**
11.1 Nominal correctness 17
11.2 Latency (REQ-PERF-001) 17
11.3 Failure / recovery 19
11.4 Anomaly policy behaviour 19
11.5 Stress / breaking-point experiment 20

12 Dashboard **21**

13 Risks, limitations, and future work **22**

14 Conclusion **23**

A Full Requirements **25**

1 Summary

This project implements a distributed cyber-physical access-control system for a single controlled boundary in a BuildSim building: a simulated population presents credentials at a card reader, an autonomous Decision Service evaluates authorization and behavioral anomaly as two independent signals, a Door Actuator carries out the resulting lock/unlock commands against BuildSim, and independent sensor processes observe the physical consequences and feed them back into the next decision. Eight Go processes plus the course-provided BuildSim run as either native binaries or Docker containers; an append-only evidence log retains every observation, decision, command, and outcome for replay and evaluation; a dashboard shows live system state.

The guiding principle threaded through every design decision is *domain separation under observable evidence*: authorization answers "is this permitted", anomaly detection answers "is this usual", and neither is allowed to silently stand in for the other anywhere in the code - not as an optimization, but because the course's own learning objectives (architectural, systems, and critical thinking) are best demonstrated by a system whose separations are structural, testable, and falsifiable, not just asserted in documentation.

Field	Value
Project title	Resilient Anomaly-Aware Building Access Control
Student	Stefan Hoglund
Use case	Single-boundary building access control with autonomous authorization and anomaly detection
Repository	`applied-ai-edge-computing` (https://github.com/stefanhoglund/building-access-control; see `README.md`)
Version / date	Final submission pass, October 2026

Scope and evolution. The system settled on a materially smaller physical footprint than a typical multi-room environmental-control use case would require - one controlled door rather than many rooms with continuous physical quantities - and used that headroom instead on distributed-systems depth: fail-closed behavior under a killed Decision Service, explicit `UNVERIFIED` outcomes when BuildSim cannot confirm an actuation, an identified missing-outcome gap when the Actuator itself cannot be reached, a genuine stress/breaking-point experiment, and a full Docker Compose deployment alongside the native one.

The autonomous component stayed deliberately rule-based (authorization precedence plus a two-factor anomaly scorer) rather than adopting a learned model: with one controlled door there is no data quantity or feature diversity sufficient to justify training one. The deterministic baseline also makes anomaly behavior transparent and directly testable, while the separate decision-policy boundary guarantees that anomaly cannot itself grant access. This is consistent with the course's own framing that sophistication is not the grading criterion (grading criteria, "Expectations and Scope").

2 Use case and building context

The use case is access control at one controlled boundary between a public zone and an employee zone in a BuildSim office floor: a reader at the boundary accepts a presented credential, and the system must decide, autonomously and in bounded time, whether to unlock the door - while also noticing when an otherwise-authorized presentation looks unusual enough to flag for a human to review later. BuildSim supplies the shared ground truth for the physical world this project does not own: building topology, the door's lock state and open/closed position, and a 3D viewer. It is explicitly *not* the authorization engine, the anomaly detector, the occupant-behavior simulator, or the historical record - this project supplies all four of those, around BuildSim's physical substrate.

The tension that makes the decision non-trivial is the same one real access-control systems face: authorization alone is necessary but not sufficient for security assurance. A credential can be valid and the access still worth flagging - an employee badging in at 3 a.m., the same badge presented three times in five seconds. Conflating "permitted" with "normal" would either over-trigger alerts on every legitimate late night, or silently miss exactly the behavior worth noticing. The project therefore treats authorization and anomaly as two independently computed signals that are only combined at the very end, by an explicit decision policy, never earlier (Section 7).

The cyber-physical feedback loop this project closes is: a simulated person's intent (Occupant & Movement Simulator) becomes a sensor observation (Card Reader) → a timestamped, stored event (Event/Data Service) → an autonomous decision (Decision Service) → a physical actuator command (Door Actuator) → an observable BuildSim state change (lock state, door position) → the Simulator causing the physical crossing conditioned on that state → independent sensors (DoorStateSensor, OccupancySensor) observing the *new* state → fed back into evidence the next decision (and the dashboard) can use. Section 8 walks this loop for one concrete scenario end-to-end.

3 Requirements

Requirements are grouped by prefix (`docs/requirements.md`, 54 IDs total) and verified in `docs/traceability-matrix.md`, which is this project's single authoritative requirement-status table (see Section 10). The table below is a representative selection by group; the full table, with implementing component, test, evidence, status, and a one-sentence rationale for all 54 IDs, is in that document.

ID	Type	Priority	Acceptance criterion	Verification
REQ-FUNC-001	Functional	Must	A DENY decision produces no UNLOCK command and the controlled door remains locked.	E2E DENY scenario
REQ-FUNC-004	Functional	Must	After an observed OPEN followed by CLOSED, the system issues a relock command.	Successful-passage E2E test
REQ-AUTH-008	Functional	Must	When no user or role permission matches, authorization resolves deterministically to DENY.	Authorization unit tests
REQ-DEC-003	Functional	Must	An anomaly classification shall never convert an otherwise unauthorized access attempt into ALLOW.	Policy-matrix unit test and anomalous-DENY scenario
REQ-DIST-004	Non-functional	Must	Outbound inter-process HTTP calls made through the shared client terminate within the configured 2 s timeout.	Decision-service failure experiment
REQ-DATA-002	Non-functional	Must	Historical evidence survives process and container restart without being reset.	Evidence-persistence restart test
REQ-RES-001	Non-functional	Must	Loss of the Decision Service shall not cause a locked door to unlock.	Decision-kill fault experiment
REQ-RES-002	Non-functional	Must	If BuildSim cannot confirm a commanded transition, the outcome is recorded as UNVERIFIED and never falsely as CONFIRMED.	BuildSim-unavailable actuator test
REQ-VIS-001	Functional	Should	Dashboard exposes current door state, latest access decision, anomaly status, occupancy, and recent evidence.	Manual dashboard verification

ID	Type	Priority	Acceptance criterion	Verification
REQ-PERF-001	Non-functional	Should	Under nominal load, 95% of access decisions shall be produced within 10 ms. Access-decision latency is defined as the wall-clock time from the Event/Data Service receiving an access-attempt observation to the Event/Data Service receiving the resulting `AccessDecision`. *Acceptance:* p95 $\leq$ 10 ms.	Latency benchmark
REQ-TEST-001	Verification	Must	A named scenario can be replayed with a recorded scenario ID and seed.	Scenario reproducibility test

The table above is a representative cross-section rather than a second authoritative requirement register. The complete set of 54 internal requirements - including type, priority, acceptance criterion, implementing component, verification method, evidence location, status, and rationale - is maintained in `docs/traceability-matrix.md`. Keeping the full matrix there avoids duplicating 54 rows in the report while preserving the requirement → design → verification traceability required by the course.

How verification works. Each requirement names the component that owns it, the automated test or manual experiment that exercises it, and the evidence directory or log where that is recorded - `docs/traceability-matrix.md` Part B is this table in full, and Part A is the course's own mandatory pass/fail checklist, verified separately. Of 54 internal requirements, **41 are PASS, 6 PARTIAL, 7 UNIMPLEMENTED** - the PARTIAL/UNIMPLEMENTED set is discussed honestly in Section 13, not hidden here.

For reporting purposes, the primarily non-functional requirements are the `REQ-DIST`, `REQ-DATA`, `REQ-RES`, and `REQ-PERF` groups, covering qualities such as process independence, timeouts, restart behavior, data retention and quality, resilience, and latency. The primarily functional and structural requirements are `REQ-FUNC`, `REQ-AUTH`, `REQ-TOPO`, `REQ-SIM`, `REQ-DEC`, `REQ-VIS`, and `REQ-BS`, describing the behavior and integration the system must provide. `REQ-TEST-*` and the mandatory course pass/fail checklist (reproduced as Part A of the traceability matrix) are kept explicitly separate: they define verification methodology and submission gates rather than runtime system behavior.

4 Architecture

4.1 Context diagram (C4 Level 1)

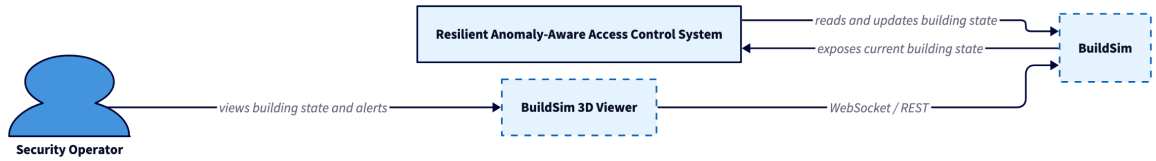


Figure 1: System context: the Resilient Anomaly-Aware Building Access Control system, its one human role (Security Operator), and the one external system it depends on (BuildSim).

The system is a single logical unit from the outside: a Security Operator views its dashboard; it reads from and writes to BuildSim over REST. Everything else - the simulated population, the authorization policy, the anomaly baseline, and the evidence store - is internal. No other external system exists; there is no cloud backend, no separate identity provider, no message broker. This is a deliberate consequence of the course's own scope guidance rather than an omission.

4.2 Container diagram (C4 Level 2)

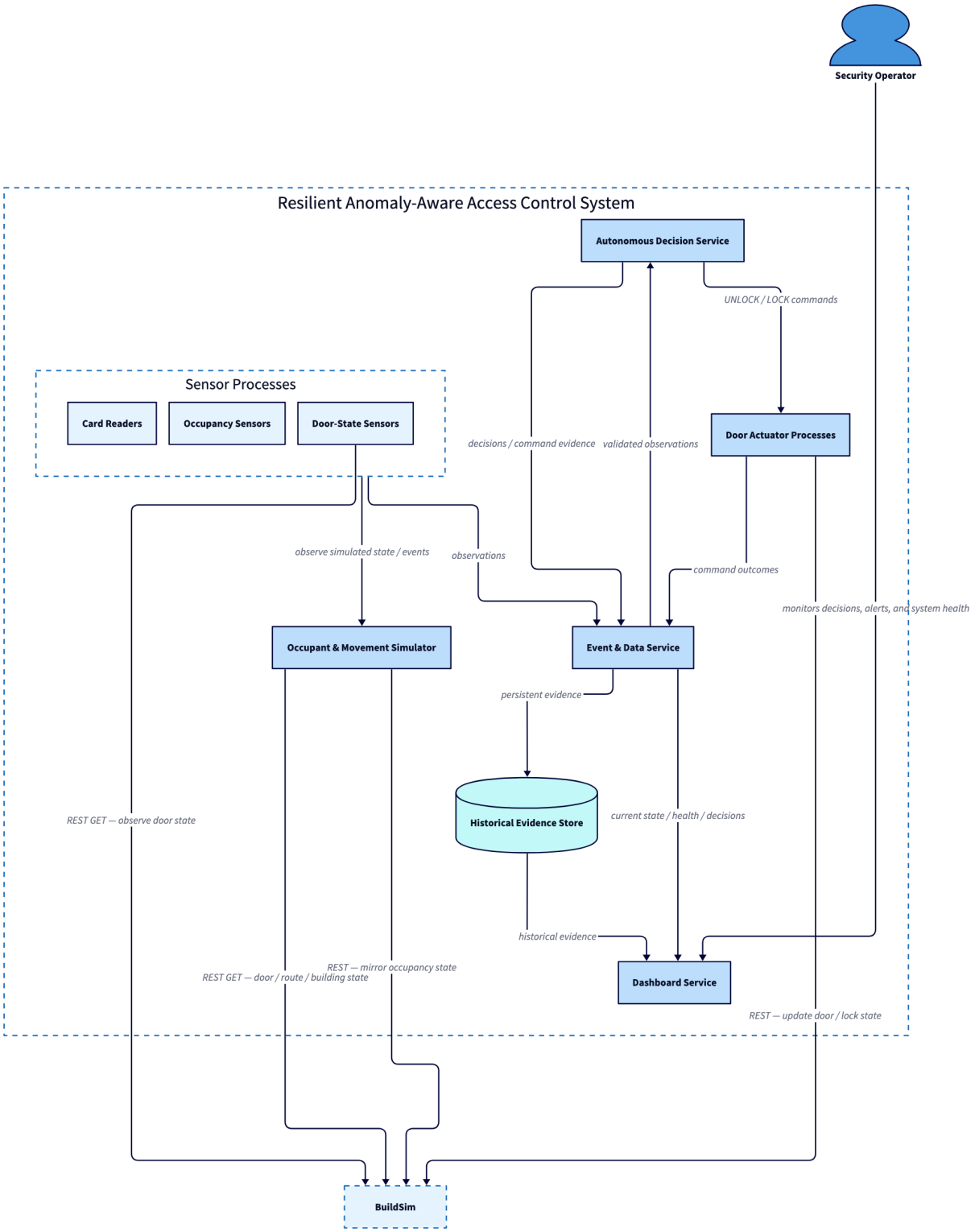


Figure 2: Container diagram: the eight independently deployable project processes and their communication paths, plus BuildSim.

Opening the system into its separately-running parts: the Occupant & Movement Simulator (owns model time and occupancy/door-position truth); three independent sensor processes (Card Reader, OccupancySensor, DoorStateSensor); the Event/Data Service (accepts, timestamps, retains, and

forwards observations, and sits on the control path - Section 5); the Decision Service (authorization, anomaly, policy, runtime assurance); the Door Actuator (issues and verifies physical commands); and the Dashboard (reads the evidence file directly). Every arrow is a real REST call or a direct file read, documented precisely in `docs/data-contracts.md`. The system is split this way because each box is independently the unit of restart, failure, and deployment the course's mandatory pass/fail criterion #1 requires - not because eight is an aesthetically pleasing number of services. Section 4.4 below names the alternatives actually considered and rejected for each major boundary.

4.3 Component diagram (C4 Level 3)

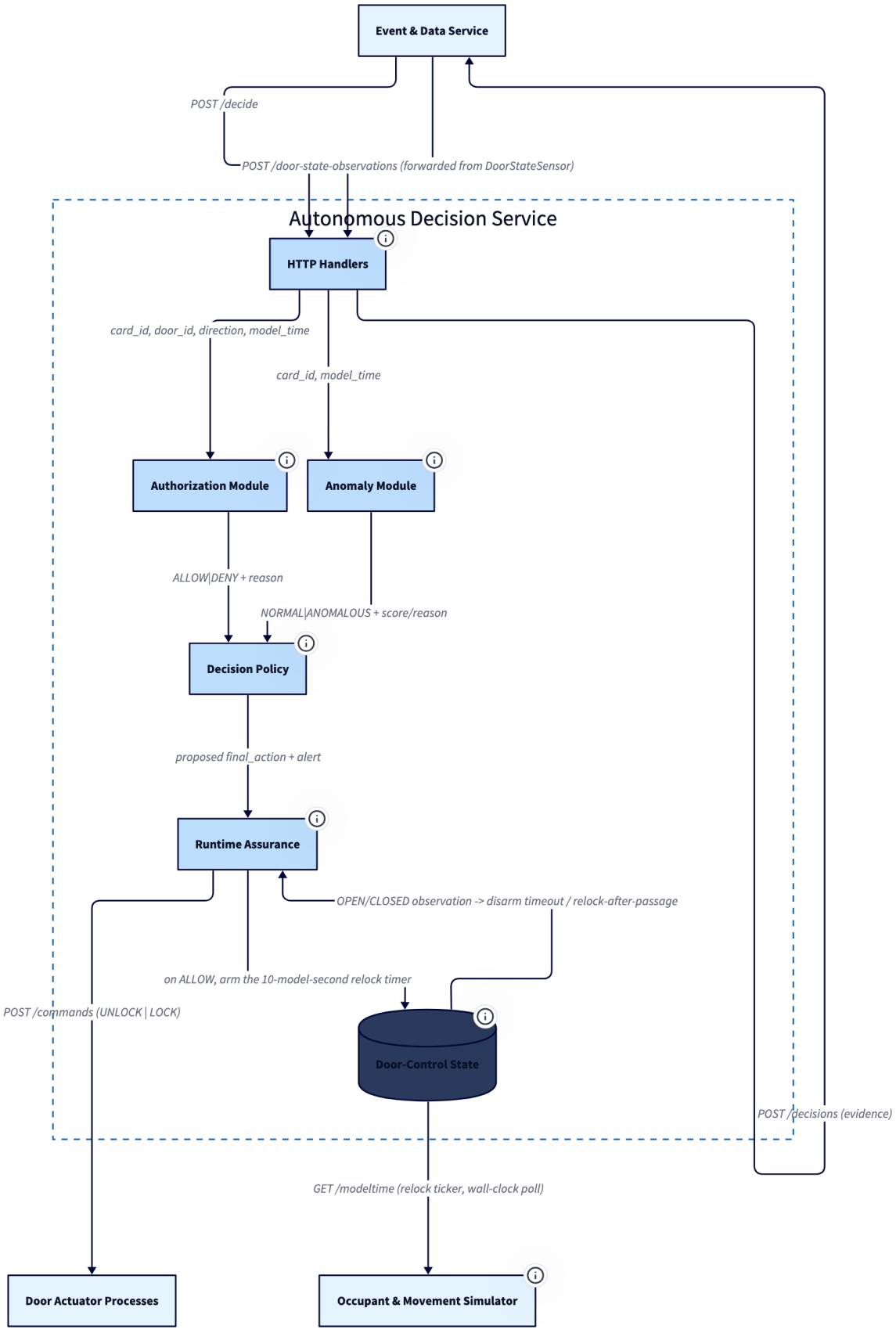


Figure 3: Component diagram: the Decision Service's internal module structure.

The Decision Service is the one container worth opening further, because its internal separation is itself a graded design decision (ADR-002). Four logically distinct modules run inside one process: **authz.go** (deterministic precedence authorization, ADR-001), **anomaly.go** (the two-factor deterministic anomaly scorer), **server.go**'s decision policy (combines the two, and *only* the two, into `final_action`), and the per-door control/runtime-assurance state (relock timers, unlock bookkeeping). The module boundary is enforced in code, not just in the diagram: `authorize()` and `anomalyTracker.score()` never read each other's return value - this is why REQ-DEC-002/003 are PASS by direct test, not by convention.

4.4 Design decisions and alternatives

Decision	Alternatives considered	Why this one
One Autonomous Decision Service process containing four logically separate modules (ADR-002)	(a) one network microservice per module (authorization/anomaly/policy/ runtime-assurance each independently deployable); (b) all decision logic folded directly into the Event/Data Service	(a) would add three more network hops, timeout surfaces, and deployment units for zero required-by-the-course benefit - the mandatory criterion asks for *an* independently deployable autonomous component, not one per internal function. (b) would blur the Event/Data Service's own single responsibility (ADR-005) and make the authorization/anomaly separation a code-review convention instead of the explicit module boundary currently enforced inside the Decision Service. The chosen option keeps the separation structurally enforced in code while avoiding unjustified network calls.
Sensor observations route through the Event/Data Service before reaching the Decision Service, not directly sensor-to-decision (ADR-005)	(a) sensors call the Decision Service directly, with the Event/Data Service only receiving a copy for storage; (b) the chosen path, Event/Data Service as the sole gate	(a) creates two sources of truth for "was this observation accepted" and would force validation or data-quality logic to live in two places or be skipped. The chosen path gives the system one authoritative location *where* malformed, duplicate, late, or stale-data handling can be implemented without duplicating that logic across sensors and the Decision Service. That handling is not complete in the current prototype (REQ-DATA-004/005 and REQ-RES-003/005 remain incomplete), but the architectural enforcement point is explicit. The same choice also makes the Historical Evidence Store's "sole writer" invariant (ADR-005) straightforward to state and verify. The cost is that the Event/Data Service is now on the critical access-control path - named explicitly as a trade-off in ADR-005 and revisited in Section 13.
REST over HTTP for every inter-process call, not a message broker	MQTT/a message broker (explicitly suggested by the course materials as a comparison point)	With nine processes, mostly request/response in shape ("decide this", "confirm this command"), and no requirement for pub/sub fan-out or offline message buffering, a broker adds an extra piece of infrastructure, deployment surface, and failure mode without an offsetting benefit. REST with bounded timeouts (`internal/httpx`, 2s default, REQ-DIST-004) is precise enough to document per-endpoint (`docs/data-contracts.md`) and simple enough that every one of this project's fault experiments could inject a failure at a well-defined point.

Decision	Alternatives considered	Why this one
Append-only JSONL file as the Historical Evidence Store, not a database	PostgreSQL/SQLite; an embedded key-value store	No query pattern in this project needs indexing, joins, or concurrent writers (ADR-005: Event/Data Service is the sole writer) - a database would add an operational dependency and a schema-migration surface for zero required capability. A plain file is trivially restart-survivable (REQ-DATA-002, directly demonstrated) and trivially inspectable by a human or a test (`evidence.ReadAll`).
Deterministic rule-based anomaly scoring, not a learned model	A trained classifier (e.g., logistic regression or an LLM-based judge) on access-time/repeat-attempt features	With one controlled door and a handful of simulated people, there is no realistic quantity or diversity of training data to justify a learned model. The deterministic baseline therefore makes anomaly behavior transparent, repeatable, and directly testable. Independently of how anomaly is scored, the decision-policy boundary ensures that neither this rule-based scorer nor a future learned replacement can grant access by itself: `final_action` is derived only from authorization (REQ-DEC-003). The course's own grading philosophy also states explicitly that more sophisticated AI does not by itself earn a higher grade.
Docker Compose, one image shared by all eight project services, plus BuildSim's own (unmodified) course-provided image	Kubernetes; one Dockerfile/image per service	Nine long-running containers that mostly talk over plain REST on one host do not need an orchestrator with its own control plane - the course explicitly discourages this (§3: "do not introduce Kubernetes ... unless explicitly required by an experiment"). Building eight separate images for eight binaries that share 100% of their source tree would only multiply build time and image count for no operational benefit; one multi-stage image with per-service `command:` overrides keeps the image count low while every container still restarts independently (verified, Section 9).

5 Interfaces and communication

Every inter-process interface is REST over HTTP/JSON, fully specified in `docs/data-contracts.md`: endpoint, method, request/response schema, and an example payload for each. Representative examples:

- `POST /observations` (Event/Data Service) - `AccessAttemptObservation` (`event_id`, `source_id`, `model_time`, `source_time`, `receive_time`, `schema_version`, `card_id`, `door_id`, `direction`) - the Event/Data Service stamps `receive_time`, persists the record, and forwards it to `POST /decide`.
- `POST /decide` (Decision Service) - the same observation; response is `AccessDecision` (`decision_id`, `authorization_result/reason`, `anomaly_result/reason/ score`, `final_action`, `alert`, `policy_version`).
- `POST /commands` (Door Actuator) - `ActuatorCommand` (`command_id`, `decision_id`, `action`, `reason`); response path produces `ActuatorOutcome` (`result` $\in$ {`CONFIRMED`, `UNVERIFIED`}, `observed_state`).

- `POST /scenario/run` (Simulator) - `{scenario, seed, count?, interval_ms?}`; blocks until every presentation the named scenario specifies has been enqueued, returns `{run_id, scenario, seed, model_start, steps_executed, card_ids}` (Section 6).

Every outbound inter-process HTTP call made through the project's shared client has a bounded timeout (`internal/httpx`, `DefaultTimeout = 2s`, REQ-DIST-004). This prevents project-controlled HTTP clients from waiting indefinitely for an unavailable peer and is what makes fail-closed behavior under a killed Decision Service (Section 11) a bounded-time property rather than an unbounded hang.

Control-path design (ADR-005). Observations are routed Sensor → Event/Data Service → Decision Service, not Sensor → Decision Service directly. This puts the Event/Data Service on the critical access-control path - a deliberate trade-off (Section 4.4): it buys one validation/evidence boundary instead of two, at the cost of one additional process the decision latency now depends on. Decision latency was measured specifically with this path included (Section 11) and remains far below threshold.

Evidence flow. The Event/Data Service is the Historical Evidence Store's sole writer (ADR-005); every observation, decision, command, and outcome is appended to `data/evidence.jsonl` as it is produced. The Dashboard and every evaluation tool read this file directly and re-read it fresh on every request - there is no separate read API, by design (one less interface to keep consistent with the write path).

BuildSim adapter. BuildSim's `PUT /api/doors` and `PUT /api/occupancy` both wholesale-replace their entire collections (confirmed from BuildSim's own source, not assumed). The `internal/buildsim.HTTPClient` adapter hides this behind a GET-modify-PUT pattern: `SetDoorState`/`SetDoorPosition` read the full current door list, mutate only the targeted door's field (preserving every other door's existing wire data, including fields this project does not otherwise use, such as `entry_node_id`), and PUT the whole collection back. The same pattern applies to room occupancy mirroring (`SetRoomOccupancy`), which also round-trips BuildSim's `aliens` field even though this project never populates it, purely so an unrelated field is never silently dropped on a PUT.

Model time vs. wall-clock time. The Occupant & Movement Simulator is the sole owner of model time (ADR-003); every other process either queries it (`GET /modeltime`) or receives it stamped on an observation, never infers it from its own clock. Model time governs scenario timing, the anomaly scorer's time-of-day factor, and logical relock timing; wall-clock time governs network timeouts, process health, and every latency/recovery-time measurement in Section 11. The two are never mixed implicitly - for example, the relock-timeout ticker wakes on a wall-clock cadence but compares against model time. Time-window authorization was designed to use model time but remains unimplemented (REQ-AUTH-005).

6 Simulation

The Occupant & Movement Simulator owns model time and simulated occupant/occupancy truth (ADR-003, ADR-006). It is deliberately small: exactly one controlled door, and a configured population of eight simulated people (`config/population.json`) spanning every role the authorization configuration defines (employee, facilities, manager, visitor, student) - behavioral diversity, not population size, is the design goal, consistent with the course's own framing that a simple, well-justified system scores higher than a technically ambitious one that cannot be clearly explained.

Discrete physical model. Unlike temperature, CO_2, or HVAC control, this access-control use case has no meaningful continuous environmental quantity that must be integrated over time. Its cyber-physical model is therefore discrete-event rather than differential-equation based.

Let $O_A(t)$ and $O_B(t)$ denote the simulated occupant counts on the two sides of the controlled boundary. For one authorized crossing from room A to room B,

$$O_A(t+1) = O_A(t) - 1, \qquad O_B(t+1) = O_B(t) + 1,$$

but only if the commanded physical state has been observed in BuildSim as `UNLOCKED`. Otherwise,

$$O_A(t+1) = O_A(t), \qquad O_B(t+1) = O_B(t).$$

The actuator therefore changes the physical condition under which the next simulated state transition is permitted. Door position and lock state are subsequently observed by independent sensor processes and fed back into the control path. This is the project's physical feedback model: actuator output constrains the next environment transition, and that transition becomes new sensor input.

Eight named, seeded, reproducible scenarios (`config/scenarios.json`, `docs/scenarios.md`) are invoked via `POST /scenario/run`: `normal-day` (a seeded random subset/order/spacing of the population), `authorized-passage`, `unauthorized-access`, `after-hours-access`, `rapid-repeat-access`, `burst-access`, and two scenarios (`decision-service-failure`, `actuator-or-buildsim-failure`) whose role is to drive a single reproducible presentation for an operator to pair with a process kill/restart - the scenario itself never performs process control, since the Simulator must not reach outside its own domain to manage other processes. Each run records `{run_id, scenario, seed, model_start, steps_executed, card_ids}` to `evidence/scenarios/runs.jsonl`, a reproducibility log kept deliberately separate from the Historical Evidence Store (ADR-005's sole-writer invariant stays about the core evidence file, not this experiment-metadata side channel).

Occupancy truth. The Simulator owns occupancy truth directly (ADR-006): on a successful crossing it decrements the source room's count and increments the destination's, and only *then* mirrors both counts to BuildSim for visualization. A denied presentation never triggers this at all - the crossing path is gated by a bounded wait for BuildSim to report the door `UNLOCKED` (REQ-SIM-004), which a DENY decision never produces, so "an occupant never crosses a locked door" holds structurally, not by a separate check.

Why simulation is intentionally simple. The purpose of this layer is reproducible evaluation, not simulation realism: the full probabilistic `BehaviorProfile` the design specification describes (arrival/dwell-time distributions, transition probabilities) was deliberately not built. What *is* built - seeded selection, order, and timing of a small, diverse population - is exactly enough to drive the required nominal, anomalous, passage, and fault-oriented scenarios reproducibly, and no more. This is reflected honestly in `docs/traceability-matrix.md`: REQ-SIM-001/002 are PARTIAL, not PASS, for exactly this reason.

BuildSim occupancy visualization limitation. BuildSim's mirrored occupant list is not authoritative identity: when a named scenario sends several different simulated people through the door over one run, BuildSim's visualization represents the destination room's nonzero count as whichever person crossed most recently, not an accumulating roster (`internal/simulator/server.go`, `mirrorOccupancy`). The Simulator's own `/occupancy` endpoint and the `OccupancyObservation` pipeline both carry the real count regardless - only the *visual* mirror simplifies, consistent with ADR-006 stating the mirror is one-way and never read back.

The named scenarios provide reproducible inputs for the nominal, anomalous, passage, and fault-oriented evaluations in Section 11: each can be replayed with the same scenario name and recorded seed. The throughput stress experiment is deliberately separate from this path: `cmd/stressbench`

submits directly to `POST /observations` so that the capacity of the Event/Data and decision pipeline can be measured independently of the Card Reader's single polling loop.

7 Autonomous decision-making and data pipeline

7.1 The autonomous service

The Decision Service combines two independently computed signals into one policy decision (Section 4.3's component diagram):

Authorization (`internal/decision/authz.go`) is a pure function implementing ADR-001's deterministic precedence chain: invalid credential → DENY; explicit user DENY → DENY; explicit user ALLOW → ALLOW; matching role DENY → DENY; matching role ALLOW → ALLOW; no match → DENY. Every step is independently unit-tested, including every pairwise conflict between a role permission and a user override.

Anomaly detection (`internal/decision/anomaly.go`) is a deterministic rule-based baseline summing two weighted factors: unusual access time (model-time hour outside 07:00-19:00, weight 0.6) and repeated attempts (the same card within 5 model seconds of its own last attempt, weight 0.6), classified `ANOMALOUS` at a summed score ≥ 0.5. It is given only the card ID and model time - never the authorization result - and its output (`anomaly_result`, `anomaly_reason`, `anomaly_score`) is recorded on every decision regardless of classification.

Decision policy combines them in exactly one place (`handleDecide`): `final_action` is set *solely* from the authorization result; the anomaly result only ever sets a separate `alert` boolean. Nothing in the function signature or call graph gives the policy step access to override this - REQ-DEC-003 ("an anomaly score alone shall not grant access that is otherwise unauthorized") is therefore a structural property, verified directly by `TestDecide_Unauthorized_DeniedRegardlessOfAnomaly` rather than inferred from code review.

Runtime assurance is the per-door control state and relock lifecycle: an ALLOW's unlock starts a bounded no-open timer (10 model seconds, REQ-FUNC-003); a completed passage (observed via DoorStateSensor's OPEN then CLOSED) triggers an immediate relock instead. Both paths converge on the same control-path invariant: through this project's own control path, an unlock command is issued only as the direct, logged consequence of an ALLOW decision. External mutation of BuildSim state is outside that invariant because BuildSim is a shared external dependency rather than state exclusively owned by the Decision Service.

Why the anomaly baseline is deliberately simple. Beyond the ML justification already given in Section 4.4, a third candidate factor the design specification names (unusual destination/zone) was deliberately not implemented: with exactly one controlled door, there is nothing for a destination-unusualness factor to vary against, so building it now would be untestable rather than genuinely evaluated. The two factors that *are* implemented were chosen because both have a directly constructible positive and negative test case (Section 11).

7.2 The data pipeline

The Event/Data Service (`internal/eventdata`) is the pipeline's single entry point: it stamps `receive_time`, appends the record to the evidence store, and forwards observations to the Decision Service (ADR-005). The Historical Evidence Store (`internal/evidence`) is a plain append-only JSONL file, written only by the Event/Data Service, read by the Dashboard and every evaluation tool. Every retained record type (observation, decision, command, outcome, door-state observation, occupancy observation, door event) carries a stable `event_id`/`command_id`/ `decision_id` plus `model_time`, so any record can be traced back through the chain that produced it (REQ-TEST-002, PASS). Retention is indefinite within one run (no pruning) - the file is small enough across this

project's whole evaluation campaign (a few hundred kilobytes) that retention policy was never a design pressure.

8 Behaviour

8.1 Key scenario: successful passage (dynamic diagram)

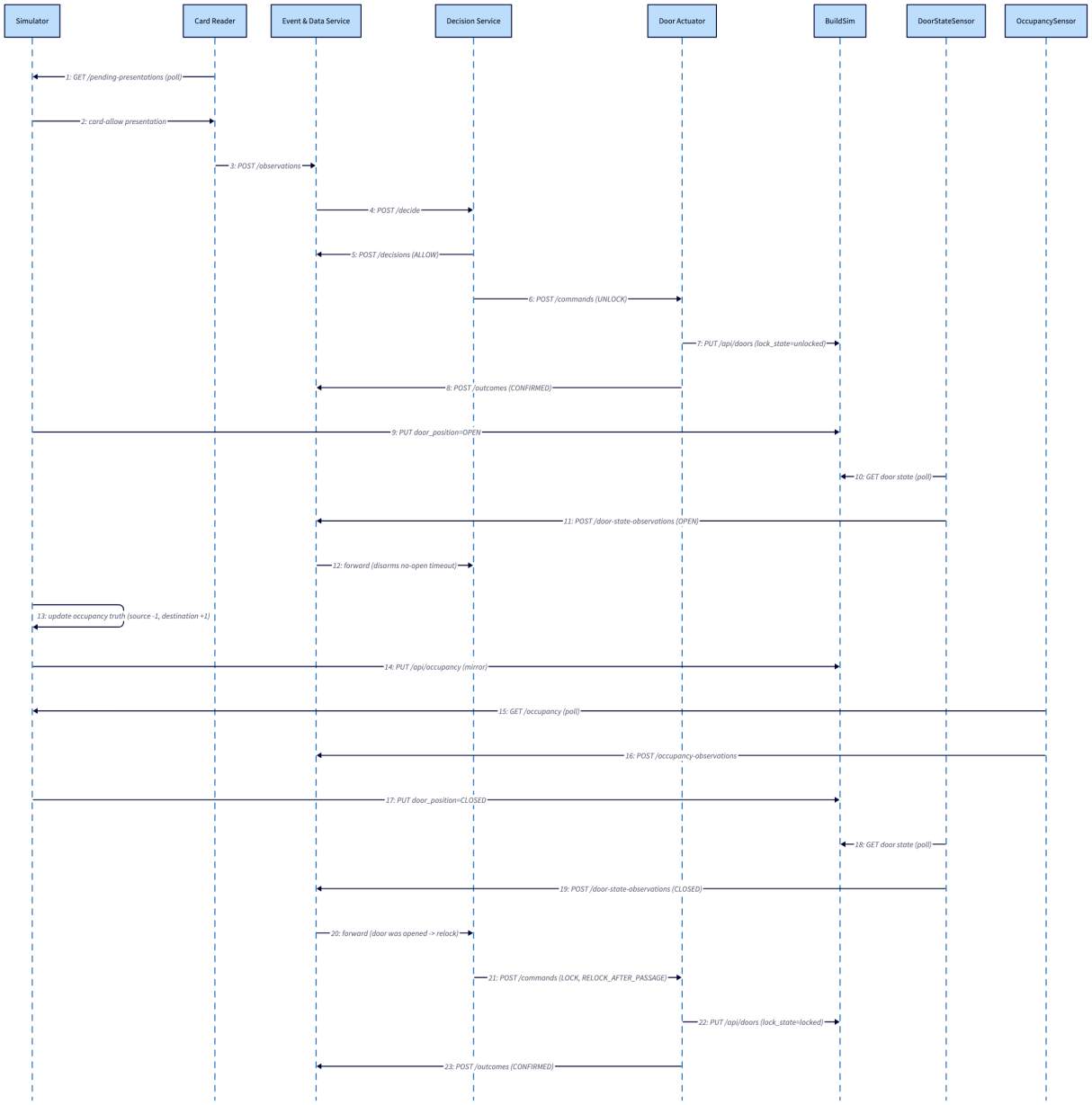


Figure 4: Dynamic diagram: the successful-passage scenario, with numbered interactions among the runtime components participating in the control loop.

Card Reader polls the Simulator (1-2), posts the observation to Event/Data (3), which forwards it to Decision (4). Decision records an ALLOW (5) and issues an UNLOCK to the Actuator (6), which writes to BuildSim and records a CONFIRMED outcome (7-8). The Simulator - watching for exactly this UNLOCKED state - opens the door in BuildSim (9); DoorStateSensor, polling independently, observes OPEN and forwards it, disarming the no-open timer (10-12). The Simulator then updates occupancy truth and mirrors it to BuildSim (13-14); OccupancySensor

observes and records the new count (15-16). The Simulator closes the door (17); DoorStateSensor observes CLOSED (18-19); Decision, informed of the completed passage, issues a relock (20-23). Every step in this sequence is machine-checked by `test/e2e/passage_test.go` and was reproduced against the real BuildSim binary and the Docker Compose stack during final verification (Section 9).

8.2 Door-control lifecycle (state diagram)



Figure 5: State diagram: the door-control lifecycle the Decision Service drives and reacts to.

From LOCKED, an ALLOW moves to UNLOCKED. From there, two paths exist: DoorStateSensor observes OPEN (disarming the no-open timer) and the state machine proceeds to CLOSED-then-LOCKED via `RELOCK_AFTER_PASSAGE`; or 10 model seconds elapse with no OPEN observed, producing a `DOOR_NOT_OPENED` event and an immediate LOCK via `RELOCK_TIMEOUT`. A third, orthogonal state, `UNVERIFIED`, is reachable from either LOCKED or UNLOCKED whenever a commanded transition's BuildSim confirmation fails - it is a confidence qualifier on the commanded state, not a peer physical state, and it resolves back to LOCKED or UNLOCKED once a later command succeeds. DENY never enters this machine at all: it produces no command, so the door's logical state never leaves LOCKED (REQ-FUNC-001).

9 Deployment

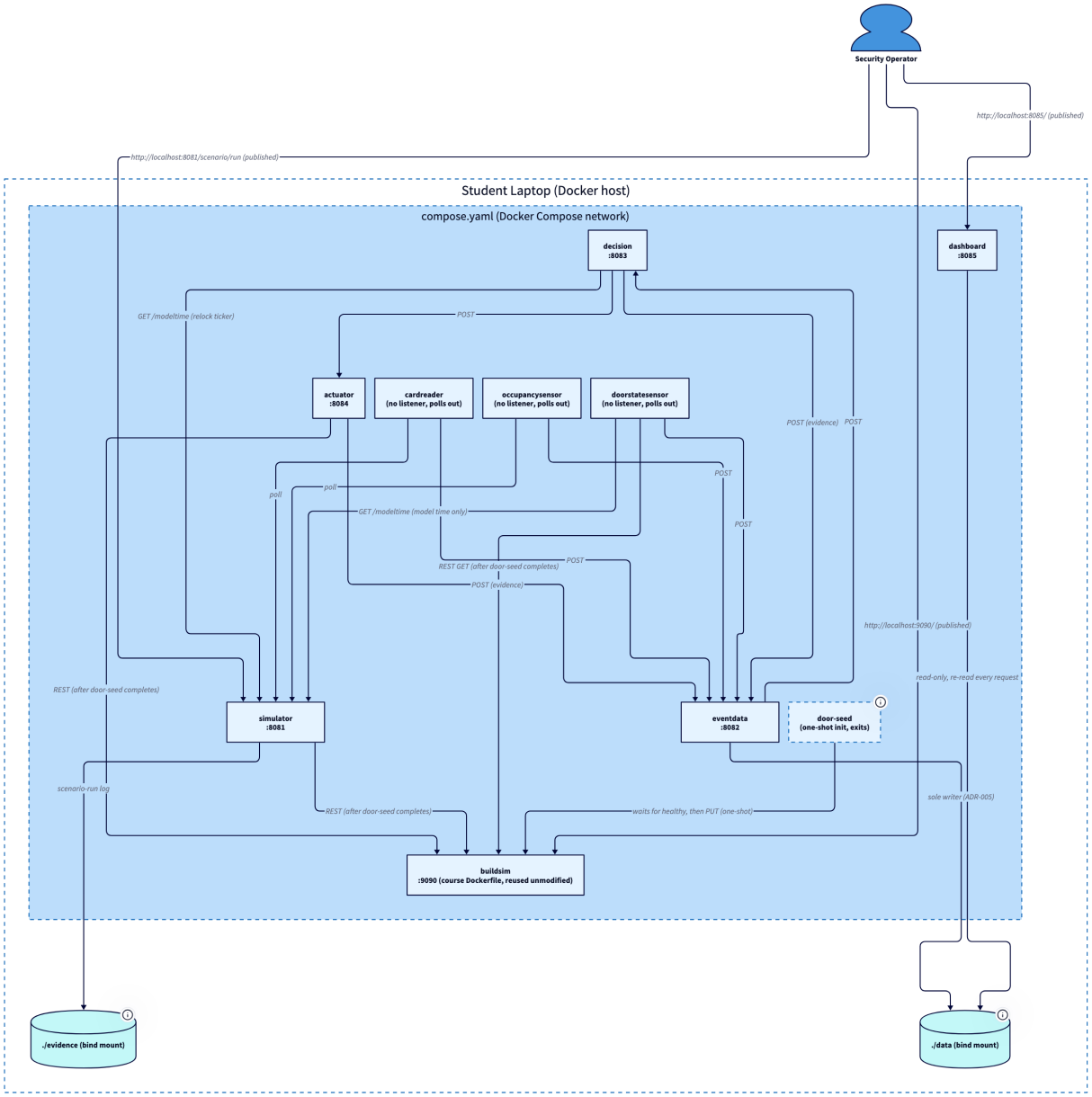


Figure 6: Deployment diagram: the Docker Compose topology - ten Compose services (nine long-running, one one-shot init step), internal DNS names, published ports, and the two persistent bind mounts.

Two supported ways to run the system exist side by side, both verified in this pass: native (`scripts/run-local.sh` + `make build/run/stop/status`, eight real Go binaries plus the real BuildSim binary on the host) and Docker Compose (`compose.yaml` at the repository root). Neither is a thin wrapper around the other - both run the exact same binaries, configured only through command-line flags.

Placement reasoning. All processes run on one laptop in the course deployment, but their logical placement still reflects an edge architecture. In a real building, the Card Reader, Event/Data Service, Decision Service, Door Actuator, and DoorStateSensor belong on local edge compute because authorization and safe door control must continue even when external connectivity is unavailable, and their latency lies on the physical control path. The Simulator is a course/test-time

substitute for the physical environment and would not exist in the same form in a production building.

Historical evidence could be replicated asynchronously to central or cloud storage, and the operator dashboard could be hosted remotely, because neither is required to complete an access decision. No GPU or external compute is justified for the implemented decision logic: the deterministic authorization and anomaly functions require negligible CPU and measured sub-millisecond decision time under nominal conditions. Keeping the safety-critical path local therefore minimizes dependency and latency while leaving monitoring and long-term analytics free to move off the edge.

`compose.yaml` **defines ten services in total**: the eight project application processes, each its own container, sharing one multi-stage `Dockerfile`-built image (`access-control:latest`) to avoid rebuilding an identical source tree eight times, each started with a different `command:` override; BuildSim's own, unmodified, course-provided image (one service); and a one-shot `door-seed` init container (one service) that (re-)creates door-1 as LOCKED/CLOSED on every fresh stack. Nine of the ten are expected to reach a steady running/healthy state; `door-seed` is expected to *exit* (status `0`) once door-1 is seeded, which is why `docker compose ps`'s default view - and every "nine" count elsewhere in this section - excludes it, not because it is uncounted but because it is not meant to still be running. **Independent restartability** was verified directly, not assumed: `docker compose restart decision` followed immediately by a fresh access attempt, and `docker compose stop/start actuator` around a presentation while it was down, both recovered correctly with the other eight containers continuing unaffected (`evidence/final-verification/`).

Addressing is entirely Compose-file configuration, not application code: every base-URL/listen-address flag already defaulted to `127.0.0.1/localhost` for native use; in Compose the same flags are passed Compose's internal DNS names (`buildsim`, `simulator`, `eventdata`, `decision`, `actuator`, `cardreader`, `occupancysensor`, `doorstatesensor`, `dashboard`). No `cmd/*/main.go` file needed a single change for this.

Persistent evidence. Two host bind mounts (`./data:/app/data`, `./evidence:/app/evidence`) carry `data/evidence.jsonl` and `evidence/scenarios/runs.jsonl`. Native default relative paths resolve onto them unchanged (the image's `WORKDIR` is `/app`) - no path flag override was needed either. Verified directly: both files survive a full `docker compose restart` of every container, growing normally rather than resetting.

A genuine deployment-layer defect found and fixed during this pass: a bulk `docker compose restart` (every container at once, not `up`) could race `door-seed`'s one-shot re-seed against BuildSim's own container restart, because `depends_on: condition: service_healthy` is honored by `up` but not by a bulk `restart` - observed directly as `curl: (6) Could not resolve host: buildsim`, leaving door-1 unseeded. The smallest correct fix was applied: `door-seed`'s curl call now retries (`-retry-all-errors`), which tolerates the race regardless of which restart path triggered it without changing normal startup timing. Reproduced twice before the fix, confirmed fixed twice after (`evidence/final-verification/README.md`, `docs/deployment.md` §3). This is named here deliberately, not only in the risks section.

10 Test plan and verification

The table below summarizes verification by test level. Every internal requirement has a requirement-specific verification approach and evidence location in `docs/traceability-matrix.md`; that document is the validation end of the requirement → design → verification thread. Reproducing the full 54-row matrix here would duplicate the authoritative artifact, so this section instead summarizes the kinds of verification used and the final regression results.

Level	Coverage
Unit tests	`internal/decision` (authorization precedence - 13 tests, anomaly scorer in isolation - 6 tests, full policy matrix - 3 tests), `internal/actuator` (confirmation, deduplication, BuildSim-unavailable - 4 tests), `internal/modeltime` (format/ parse/advance determinism - 4 tests), `internal/simulator` (seed reproducibility - 1 test).
End-to-end tests	`test/e2e/`: ALLOW, DENY, relock-timeout, successful-passage (`FakeBuildSimClient`), decision fail-closed/restart/recover, and five named-scenario tests exercising the real `POST /scenario/run` interface.
Integration test	`test/integration/realbuildsim_test.go`: the same ALLOW/relock sequence against the real, course-provided BuildSim binary, not the fake.
Scenario tests	Authorized crossing, denied access, after-hours anomaly, rapid-repeat anomaly, burst count - all via the production `POST /scenario/run` interface, not a test-only backdoor.
Docker Compose verification	A full manual pass against the real containerized stack: all services healthy, door-1 seeded, all four scenario types, a container restart, an actuator-stop fault, dashboard reachability, and evidence persistence - repeated at the end of this pass with a fresh build (`evidence/final-verification/`).

Results, this pass (2026-10-10): `gofmt -l .` clean; `go vet ./...` clean; `go test ./...` and `-race` green across every package; `go vet -tags=integration ./...` clean; `go test -tags=integration ./test/integration/...` green against the real BuildSim binary; all three `config/*.json` files valid JSON; `docker compose config/build/up` all succeed; all nine long-running Compose services reach running/healthy (the tenth, `door-seed`, exits as expected - Section 9). Full logs, exact commands, and timestamps: `evidence/final-verification/`.

Requirement traceability is `docs/traceability-matrix.md` (Section 3): every one of the 54 internal requirements and all 9 mandatory course pass/fail criteria are mapped to a specific test or experiment and an evidence location there, not merely asserted.

11 Evaluation

This section is evidence, not assertion: every number below is read from a committed evidence file (`docs/evaluation-results.md` is the same content in prose-and-table form with full source citations).

11.1 Nominal correctness

Every functional scenario in `docs/scenarios.md` was exercised and produced the expected result: ALLOW/DENY, the full successful-passage sequence, the no-open relock timeout, and reproducible seeded selection of a population subset (`normal-day`, seed 42, consistently selecting `[card-henry, card-frank, card-dave, card-erin, card-carol]` across repeated runs). See `docs/evaluation-results.md` §1.

11.2 Latency (REQ-PERF-001)

$n = 200$ presentations, in-process over loopback (`cmd/latencybench`), wall-clock.

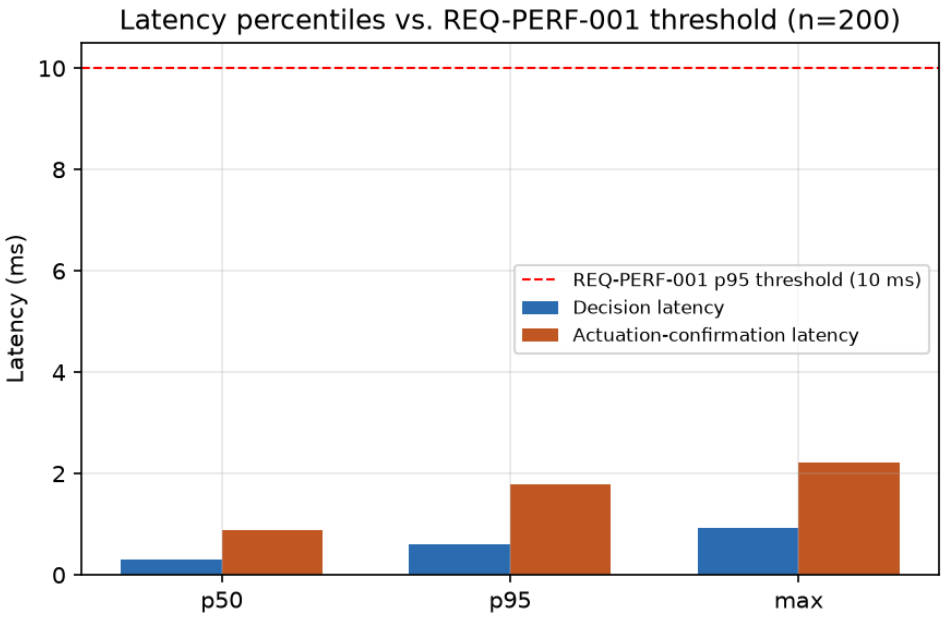


Figure 7: Decision and actuation-confirmation latency percentiles against the REQ-PERF-001 threshold.

Metric	**p50**	**p95**	**max**	**Threshold**
Decision latency (ms)	0.298	0.601	0.928	p95 $\leq$ 10 ms - met
Actuation-confirmation latency (ms)	0.884	1.791	2.21	no formal threshold; secondary metric

The threshold (p95 $\leq$ 10 ms) was set *after* this exploratory baseline was measured, deliberately - high enough above the measured 0.601 ms to accommodate ordinary scheduling jitter and modest load growth, while still imposing a materially tighter engineering bound than the project's 2 s network timeout. It should therefore be read as a post-baseline engineering requirement rather than a preregistered performance target. Caveat, both latency metrics were measured against `FakeBuildSimClient`, not the real BuildSim HTTP adapter's GET-modify-PUT round trip; the physical actuation path's true latency against the real adapter was not separately benchmarked.

11.3 Failure / recovery



Figure 8: Decision Service restart recovery time across 10 fresh runs.

Killing the Decision Service mid-attempt produces no decision and no command - a locked door stays locked (REQ-RES-001) - confirmed both automatically and against a real killed OS process. Restarting it as a fresh process, with no carried-over state, recovers correctly; 10 runs captured during this pass measured **min 10.83 ms, mean 18.8 ms, max 32.26 ms** from restart to the first correct ALLOW/UNLOCK.

Two actuator-side failure cases must be distinguished. When BuildSim is unavailable while the Door Actuator itself remains running, the Actuator cannot verify the physical state transition and therefore records an `UNVERIFIED` outcome with no confirmed `observed_state`, never a false `CONFIRMED` (REQ-RES-002). Once BuildSim returns, later commands can again be confirmed normally.

A different failure occurs when the Actuator itself is unreachable from the Decision Service. In that case the command can be recorded as issued by Decision, but delivery fails before the Actuator can produce any outcome record. This missing-outcome case was exposed during stress testing and remains an identified limitation rather than being misclassified as `UNVERIFIED`.

11.4 Anomaly policy behaviour

Scenario	Authorization	Anomaly	Final action	Alert
Normal authorized (day)	ALLOW/ ROLE_ALLOW	NORMAL	ALLOW	false
Anomalous but authorized (night)	ALLOW/ ROLE_ALLOW	ANOMALOUS/ UNUSUAL_ACCESS_TIME	ALLOW	true
Unauthorized (also night)	DENY/ NO_PERMISSION	ANOMALOUS/ UNUSUAL_ACCESS_TIME	DENY	true
Rapid repeat, 2nd/3rd attempt	ALLOW/ ROLE_ALLOW	ANOMALOUS/ REPEATED_ATTEMPT	ALLOW	true

Row 3 is the direct proof of REQ-DEC-003: the anomaly classification is reported honestly

regardless, but it never changes `final_action` - a DENY stays a DENY even when it is also, separately, anomalous.

11.5 Stress / breaking-point experiment

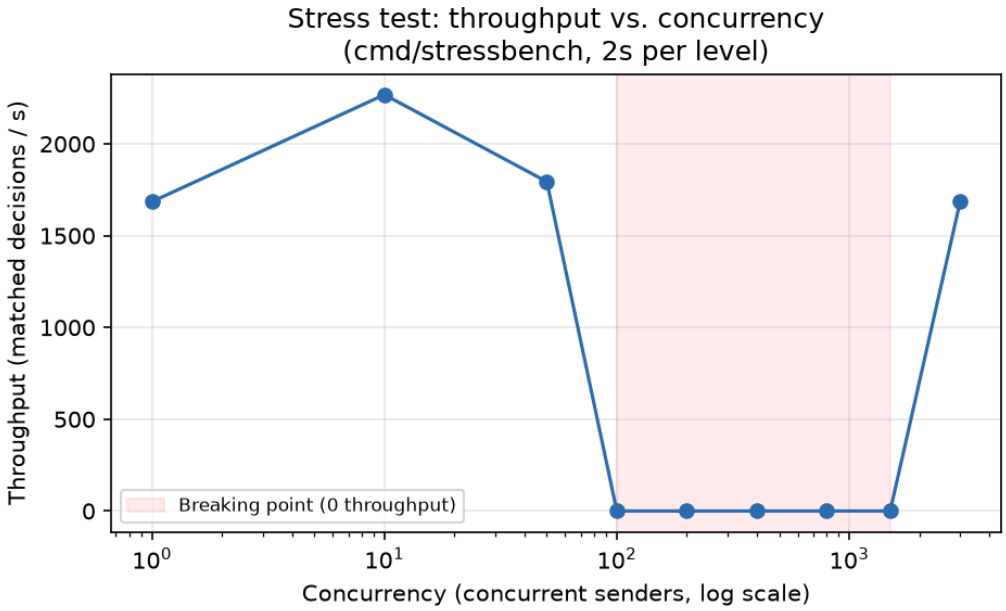


Figure 9: Throughput collapses to zero between concurrency 50 and 100, partially recovering at 3000.

`cmd/stressbench`, concurrency swept 1-3000, firing directly at `POST /observations` (bypassing the Card Reader's own single-poll-loop bottleneck deliberately, to measure the pipeline's own limit, not that unrelated one).

Concurrency	Throughput (req/s)	p95 (ms)	Err.	Interpretation
1	1685	0.20	0%	Nominal
10	2268	1.86	0%	Nominal
50	1794	4.50	3.1%	Early degradation
100	**0**	-	**100%**	**Breaking point**
200-1500	0	-	100%	Total failure persists
3000	1687	1877.77	72.9%	Partial, unstable recovery

Each concurrency level in this sweep was sampled once. The non-monotonic partial recovery observed at concurrency 3000 should therefore be interpreted as evidence of unstable failure behavior, not as a repeatable recovery threshold. Repeated trials at each concurrency level would be required to estimate variance and establish a statistically stable breaking region.

Failure mechanism localized to the production HTTP path. The Decision Service's own log reports `dial tcp ...: can't assign requested address` on outbound calls to the Door Actuator under high concurrency. Each access attempt fans out into approximately four internal HTTP calls, so high concurrency creates substantial short-lived socket and connection pressure. The project uses Go's default HTTP transport, which retains only a small number of idle keep-alive connections per host; limited connection reuse can therefore contribute to additional connection churn.

An artifact in the first benchmark sweep - response bodies that were not fully drained by the benchmark client - was corrected and the sweep was re-run. The production-side failure persisted, ruling out that benchmark artifact as the explanation. The precise relative contribution of HTTP connection reuse, ephemeral-port exhaustion, and other local socket limits was not isolated further, so the result is reported as a localized connection/socket-pressure bottleneck rather than as a fully controlled causal attribution. **It was deliberately not optimized in this pass**: the purpose of the experiment was to locate and document the system's breaking behavior, not move the threshold (Section 13).

A second, previously-unknown gap this experiment surfaced: when Decision→Actuator delivery itself fails (distinct from Actuator→BuildSim failing, which REQ-RES-002 already covers), a command is recorded as issued with **no outcome ever following** - worse than `UNVERIFIED`, which is at least a record that something was attempted. Not fixed; named here and in Section 13.

12 Dashboard

The dashboard (`internal/dashboard`, `cmd/dashboard`) reads the evidence file directly and re-reads it fresh on every request - no separate API, no cached/stale view. It shows: the controlled door's `lock_state` and `door_position` with a verification indicator (CONFIRMED vs. UNVERIFIED); the latest access decision, including `authorization_reason`, `anomaly_result/reason`, and `alert`; destination-room occupancy; a per-source-ID "component health" panel derived purely from each source's most recent evidence timestamp (explicitly *not* a real heartbeat protocol - stated as such in the code, not oversold); and a table of recent evidence records. It auto-refreshes every two seconds via a plain `<meta http-equiv="refresh">`, deliberately avoiding a WebSocket/SSE layer the system has no other use for. Reachable at `http://localhost:8085/` both natively and via Docker Compose, confirmed live during final verification.

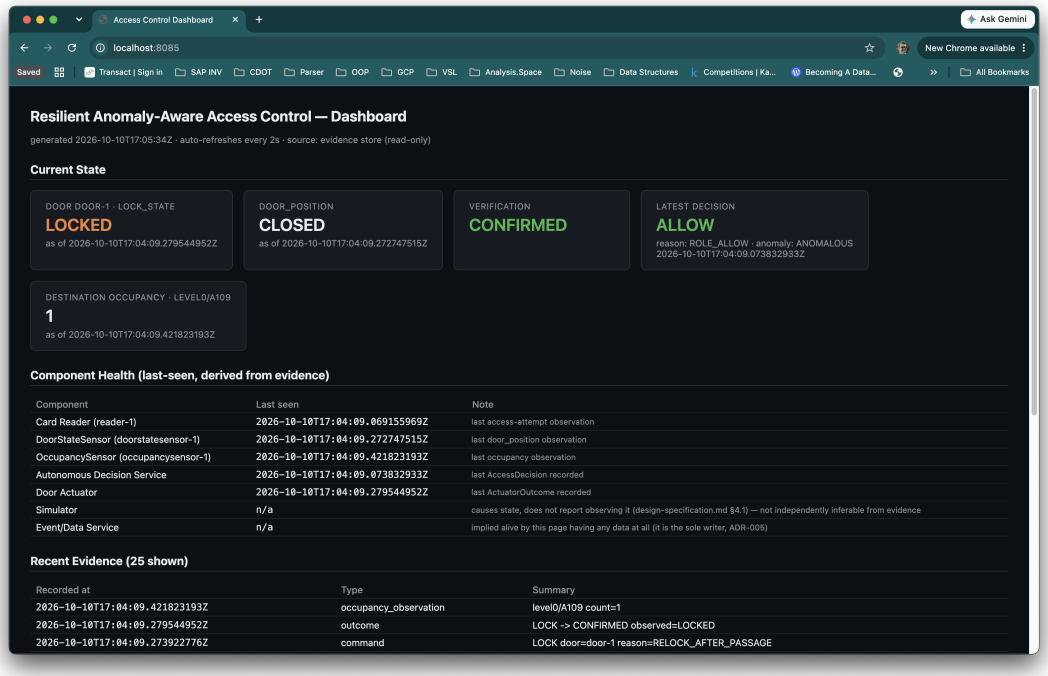


Figure 10: Operational dashboard showing live door state, recent access decisions, anomaly status, occupancy, and retained evidence.

13 Risks, limitations, and future work

- **One controlled, representative door.** The entire architecture (reader/actuator/door-state binding, zones, topology) generalizes to many doors without a redesign (REQ-TOPO-002/003 are already topology-generic), but only one was ever built or tested. What would change at 100 doors or multiple buildings: the Event/Data Service's single evidence file would need partitioning or a real store (it was never meant to scale past this project's evidence volume); the Decision Service's per-door control state (already a map keyed by door ID) would need to handle many more concurrent entries under real contention, not just the occasional overlapping presentation found during this pass (below); and BuildSim's own wholesale-replace PUT semantics would become a genuine throughput concern at that scale, not just an adapter-design curiosity.
- **7 of 54 internal requirements are unimplemented**, consistently named across this project's whole development, not discovered late: device re-registration after restart (REQ-DIST-005/006 - no `/api/equipment` flow exists); and data-quality fault handling (REQ-AUTH-005 time windows, REQ-DATA-004/005 duplicate/late detection, REQ-RES-003/005 staleness). 6 more are PARTIAL. Full list and rationale: `docs/traceability-matrix.md`.
- **Deterministic anomaly baseline, not a learned model** - justified in Section 7, but it means the system cannot adapt to behavior patterns not anticipated by its two fixed factors, and its thresholds (0.6/0.6/0.5, the time window, the repeat window) are a defensible starting point, not derived from a real evaluation dataset - none exists for this project.
- **No probabilistic crowd simulation.** Section 6 states this as a deliberate scope choice, not an oversight: REQ-SIM-001/002 are PARTIAL for exactly this reason.
- **BuildSim's occupancy mirror is not authoritative identity** - Section 6's visualization limitation; the Simulator's own count and the `OccupancyObservation` pipeline remain correct regardless.
- **BuildSim is a shared dependency across three independent processes** (Actuator, Simulator, DoorStateSensor) - a single point of failure for all three at once, named explicitly when discovered (`evidence/actuator-failure/README.md`) rather than worked around silently.
- **Connection/socket-pressure stress bottleneck**, localized to the production HTTP path but not reduced to a single fully controlled cause (Section 11). The first engineering response would be to tune and instrument `internal/httpx` connection reuse/pooling and repeat the experiment while observing socket and ephemeral-port behavior. This was deliberately not built in this pass because the experiment's purpose was to identify and characterize the limit rather than move it.
- **Decision→Actuator missing-outcome failure mode**, discovered by the stress experiment, not fixed: a command can be recorded with no resolution at all if delivery to the Actuator itself fails.
- **Single-pending-per-door control state can be clobbered by overlapping presentations.** Discovered during Docker verification when a rapid-repeat scenario's trailing presentations overlapped a fresh restart-recovery attempt on the same door: the per-door bookkeeping (keyed by `door_id`, not `decision_id`) can lose track of an earlier unlock's relock obligation. A clean, isolated request immediately afterward completed correctly, confirming this is a pre-existing property of the single-slot design, not a restart-recovery regression. Not fixed - redesigning per-door state to be per-decision or queued is out of scope for this pass.

- **Only selected failure domains were tested.** Decision Service and the Actuator↔BuildSim link were fault-tested explicitly; Card Reader/OccupancySensor/DoorStateSensor crash-recovery is architecturally identical (same restart pattern, same "unrelated processes unaffected" property) but was not separately exercised, to avoid repeating the same proof three more times for diminishing evidentiary value.
- **Actuation-confirmation latency caveat**: measured against `FakeBuildSimClient`, not the real BuildSim adapter's GET-modify-PUT round trip (Section 11).
- **Deferred stale/late/duplicate observation handling** (REQ-DATA-004/005, REQ-RES-003/005) - no freshness, ordering, or deduplication logic exists anywhere in the pipeline yet.
- **No BuildSim device/equipment re-registration flow** (REQ-DIST-005/006) - the real adapter talks to `/api/doors` directly; a restarted sensor/actuator process resumes polling/commanding immediately, but there is no explicit registration handshake to lose or repeat.
- **A genuine deployment defect was found and fixed, not just documented**, within the scope the task allowed for a small, safe correction: the `door-seed` vs. bulk-restart race (Section 9).

If more time existed, data-quality fault handling (duplicate/late/malformed detection in the Event/Data Service) is the single highest-value remaining item: it is explicitly named in the grading criteria's Dimension D examples, cheap relative to full behavioral-profile simulation (no new process, just validation logic in an already-existing service), and would produce new, easy-to-demonstrate evidence using the exact same capture pattern already established four times over in this project (`docs/final-report-gap-analysis.md`).

14 Conclusion

The original question was whether an autonomous edge system can make safe building-access decisions while remaining robust to imperfect observations, communication failures, and component failures. Within the subset actually tested - nominal observations, anomalous access behavior, Decision Service failure, BuildSim failure, and load-induced communication failure - the prototype demonstrates that it can, with explicitly identified limits. Robustness to malformed, stale, duplicate, or out-of-order observations remains unimplemented and is therefore outside the demonstrated claim.

Safe within the implemented control path: an invalid or disabled card never produces an authorized unlock; a DENY produces no unlock command; anomaly classification cannot override a denial; a failed Decision Service leaves an already-locked door locked; and when the Door Actuator is able to attempt a command but BuildSim cannot confirm the result, the outcome is recorded as `UNVERIFIED`, never falsely as `CONFIRMED`. Actuator-level command deduplication also prevents repeated delivery of the same command from causing duplicate physical effects.

Robust to the failures that were exercised: a killed Decision Service recovered in 10.83-32.26 ms across ten measured runs with no carried-over process state. An unavailable BuildSim produced `UNVERIFIED` actuator outcomes and recovered once restored. A distinct unresolved failure was also exposed: if the Decision Service cannot reach the Actuator itself, a command may be recorded without any subsequent outcome. The Docker architecture keeps all eight project services independently deployable and restartable; Decision Service and Actuator restart/failure behavior were exercised directly while unrelated services continued running. A separate deployment-layer race between BuildSim startup and the one-shot `door-seed` service was found during verification, fixed, and re-tested rather than merely documented.

Within explicit limits: the system was stress-tested to an actual breaking region rather than only under nominal load. Production logs localized the failure to outbound HTTP/socket pressure in the Decision-to-Actuator path after a benchmark-client artifact had been eliminated, although the exact contribution of connection reuse, ephemeral-port limits, and related host-level effects was not isolated further. The same experiment exposed the previously unknown Decision-to-Actuator missing-outcome case. Seven of 54 internal requirements remain unimplemented and six more are partial; these gaps are stated consistently in the traceability matrix and in this report. The anomaly baseline is deliberately simple and cannot adapt beyond its two fixed factors without further work.

The project therefore does not claim completeness or general robustness to every form of imperfect data or failure. Its demonstrated claim is narrower: within the tested prototype envelope, the architecture makes the control path, safety decisions, evidence trail, failure behavior, and known limits explicit enough to test, reproduce, and explain from evidence.

A Full Requirements

This appendix reproduces the complete set of 54 internal project requirements. The requirements are grouped by their stable identifiers and preserve the normative requirement statements and acceptance criteria from the architecture baseline. Final implementation status, verification evidence, and rationale are maintained in `docs/traceability-matrix.md`; development-time status notes are intentionally not duplicated here.

The mandatory course pass/fail checklist is kept separate from these internal system requirements because it defines assessment gates rather than runtime system behaviour.

Requirement ID	Requirement and acceptance criteria
Core functional requirements	
`REQ-FUNC-001`	The system shall deny an access attempt when no valid permission exists for the presented card, requested controlled boundary, and model time. *Acceptance:* • authorization decision is DENY • door remains locked • occupant does not cross the controlled boundary • DENY decision and reason are recorded
`REQ-FUNC-002`	The system shall allow an access attempt when the presented card is active and a valid permission exists for the requested controlled boundary at model time. *Acceptance:* • authorization decision is ALLOW • actuator receives an unlock command • BuildSim reflects the unlocked state • the occupant may cross the controlled boundary • decision and outcome are recorded
`REQ-FUNC-003`	A door that was locked before an approved access attempt shall automatically return to the locked state no later than 10 model seconds after being unlocked if the door has not been opened. *Acceptance:* • initial state is LOCKED • an ALLOW decision causes the door to unlock • no door-open event occurs • a DOOR_NOT_OPENED event is recorded • door returns to LOCKED within 10 model seconds
`REQ-FUNC-004`	When an authorized occupant opens and passes through a controlled door, the system shall relock the door after passage is complete. *Acceptance:* • valid access is approved • door opens • occupant crosses boundary • destination occupancy changes appropriately • door returns to LOCKED
Authorization requirements	
`REQ-AUTH-001`	Each access card shall be associated with exactly one active user at a time.
`REQ-AUTH-002`	Users shall be assigned one or more roles.
`REQ-AUTH-003`	Permissions shall be assignable at the role level.
`REQ-AUTH-004`	The system shall support user-specific permission overrides.
`REQ-AUTH-005`	Permissions may include time-of-day and day-of-week constraints evaluated against model time.
`REQ-AUTH-006`	A disabled, expired, or unknown card shall not authorize access.
`REQ-AUTH-007`	If no applicable permission exists, access shall default to DENY.
`REQ-AUTH-008`	The system shall define deterministic precedence when both role-level and user-specific permissions apply. *Acceptance:* • precedence rules are documented • the same inputs always produce the same authorization result • conflicting role/user permissions are covered by tests
Topology requirements	

Requirement ID	Requirement and acceptance criteria
REQ-TOPO-001	Rooms shall be assigned to security zones. • Initial security zones shall include: • public/open • employee • student • restricted
REQ-TOPO-002	A controlled door shall represent a directed or bidirectional transition between two BuildSim locations and their associated security zones.
REQ-TOPO-003	Each controlled direction shall be associated with: • a card reader • a door actuator • observable door state
REQ-TOPO-004	Room identity shall use a level-qualified identifier such as level0/A109.
Simulation requirements	
REQ-SIM-001	The simulator shall generate occupant movement and access behavior according to configured behavioral profiles, schedules, security-zone access patterns, and stochastic variation. *Acceptance:* • weekday arrival and departure distributions are configurable • room-transition probabilities are configurable • generated behavior is reproducible for a fixed seed • generated behavior differs for different seeds
REQ-SIM-002	Normal weekday behavior shall include structured arrival, departure, lunch, and room-use patterns with controlled stochastic variation.
REQ-SIM-003	Simulation runs shall support a configurable random seed so that runs can be reproduced.
REQ-SIM-004	The simulator shall prevent an occupant from crossing a locked controlled door unless the active scenario explicitly models forced entry.
REQ-SIM-005	When an occupant successfully crosses from one room to another, the simulator shall update the physical occupancy state of both rooms and cause subsequent occupancy observations to reflect the change. *Acceptance:* • source-room occupancy decreases by one • destination-room occupancy increases by one • no occupancy transition occurs when movement is denied
REQ-SIM-006	Actuator actions shall influence subsequent simulated behavior and sensor readings, closing the cyber-physical feedback loop.
Autonomous decision requirements	
REQ-DEC-001	At least one independently deployable component shall autonomously make decisions based on sensor observations and influence actuator behavior.
REQ-DEC-002	Authorization and anomaly detection shall be treated as separate decision signals.
REQ-DEC-003	An anomaly score alone shall not grant access that is otherwise unauthorized.
REQ-DEC-004	Every access decision shall record: • decision • reason • associated access event • model time
REQ-DEC-005	The system shall identify access behavior that differs from defined normal behavior.
REQ-DEC-006	Anomalous but authorized behavior shall be distinguishable from unauthorized behavior.
REQ-DEC-007	Every autonomous decision shall be traceable to the observations and decision signals used to produce it. *Acceptance:* • decision contains or references the triggering observation ID • authorization result is identifiable • anomaly result is identifiable where applicable • decision-policy version is identifiable
Distributed-system requirements	

Requirement ID	Requirement and acceptance criteria
REQ-DIST-001	Sensor, actuator, and autonomous decision components shall run as separate, independently deployable and restartable processes.
REQ-DIST-002	The system shall not be implemented as a single monolithic application.
REQ-DIST-003	Components shall communicate through documented network interfaces.
REQ-DIST-004	Network calls shall use bounded timeouts.
REQ-DIST-005	A restarted device process shall re-register with BuildSim and resume operation.
REQ-DIST-006	Repeated registration shall not create duplicate logical devices.
REQ-DIST-007	Repeated delivery of the same observation or command shall not cause duplicate physical side effects.
Data-pipeline requirements	
REQ-DATA-001	The system shall provide a defined data pipeline that collects sensor observations, stores them, and makes them available for monitoring, analysis, decision-making, training, or evaluation.
REQ-DATA-002	Observations, decisions, actuator commands, and relevant outcomes required for replay and evaluation shall survive application-process restarts.
REQ-DATA-003	Every retained observation shall include: • unique event identifier • source identifier • model time • source time • receive time • schema version
REQ-DATA-004	The pipeline shall detect and record duplicate observations.
REQ-DATA-005	The pipeline shall identify late or out-of-order observations.
REQ-DATA-006	Malformed observations shall be rejected from the validated data path, retained or logged as rejected evidence, and counted by pipeline telemetry.
Resilience requirements	
REQ-RES-001	If the authorization or decision service is unavailable, a locked controlled door shall remain locked.
REQ-RES-002	If an actuator command cannot be confirmed, the resulting door state shall be marked as unverified and the failure shall be recorded.
REQ-RES-003	Stale sensor observations shall not be treated as current without an explicit quality flag.
REQ-RES-004	The system shall recover from restart of a sensor, actuator, or decision process without requiring restart of unrelated processes. *Acceptance:* • restarted process re-registers/reconnects • normal processing resumes • no duplicate logical devices are created • recovery time is measured
REQ-RES-005	The system shall detect when a required sensor observation is unavailable or stale beyond its configured freshness limit. *Acceptance:* • stale/missing state is identifiable • affected decisions record degraded-data status • the system follows the documented degraded-state policy
Visualization requirements	
REQ-VIS-001	The dashboard shall visualize, for the active scenario: • current occupancy • controlled-door state • recent access decisions • active anomaly/alert state • degraded or failed component state where applicable
BuildSim integration requirements	
REQ-BS-001	BuildSim shall be treated as the authoritative source for current building topology, controlled-door state, and registered equipment state. • Simulated occupant location and occupancy truth are owned by the Occupant and Movement Simulator as defined in ADR-006.

Requirement ID	Requirement and acceptance criteria
REQ-BS-002	Historical observations and decisions shall not rely solely on BuildSim persistence.
REQ-BS-003	An accepted actuator command shall produce an observable BuildSim state change that can affect subsequent simulator behavior or sensor observations.
Performance requirements	
REQ-PERF-001	Under nominal load, 95% of access decisions shall be produced within a defined latency threshold. Access decision latency is defined as the wall-clock time from the Event/Data Service receiving an access-attempt observation to the Event/Data Service receiving the resulting AccessDecision (wall-clock, not model time; see docs/evaluation-plan.md). *Acceptance:* p95 $\leq$ 10 ms, p100 $\leq$ 50 ms. Baseline measurement (200 samples, nominal load, in-process over loopback, 2026-10-09): p50 = 0.30 ms, p95 = 0.60 ms, max = 0.93 ms – see evidence/latency/results.json. The threshold is set well above this measured baseline (headroom for scheduling jitter, GC pauses, and modest load growth) but far below the ~100 ms point at which an interactive, door-side response is perceived as non-instantaneous, so the requirement remains a real engineering constraint rather than an arbitrarily loose one chosen merely to pass. It was not chosen before this measurement existed.
Verification requirements	
REQ-TEST-001	The system shall support reproducible end-to-end scenarios using a configured scenario identifier and random seed.
REQ-TEST-002	The system shall expose enough evidence to trace an access attempt from sensor observation through decision, actuator command, and resulting BuildSim state.